

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	M PRITHIYANKA SHREE	Reg. No.:	192521144
Section / Batch:	2025	Date:	16-07-2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Q1. E-Library Search System

An e-library reduces the collection size by half after each comparison while searching for a title. Define divide-and-conquer strategy, explain how repeated halving reduces the problem size, and calculate the number of steps. Determine both the time complexity and space complexity of this search system.

Q2. Image Processing System

An image processing algorithm divides images recursively: $T(n) = 4T(n/2) + n$. Apply the Master Theorem to find the time complexity. Clearly identify parameters and case selection. Analyze how performance changes with image size.

Code of Conduct

I M PRITHYANKA SHREE(192521144) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M PRITHYANKA SHREE

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Introduction (Real-Time Application)

An **E-Library Search System** stores thousands or even millions of books, journals, and research papers. When a user searches for a particular title, searching every book one by one would take a long time.

To make searching faster, the e-library stores the book titles in **sorted order** and uses the **Binary Search** algorithm, which follows the **Divide-and-Conquer strategy**.

Instead of checking every book, the system compares the title with the middle element and eliminates half of the remaining collection after each comparison. This greatly reduces the search time.

2. Divide-and-Conquer Strategy

Divide-and-Conquer is an algorithmic technique in which a large problem is divided into smaller subproblems, each subproblem is solved independently, and the final result is obtained.

It consists of three steps:

Divide

The collection of books is divided into **two equal halves**.

Conquer

The system compares the required title with the middle title and continues searching only in the half where the title may exist.

Combine

For Binary Search, no combining step is required because only one half is searched until the title is found.

3. How Repeated Halving Reduces the Problem Size

Suppose the e-library contains **16 books**.

Initially,

$$n = 16$$

After each comparison, the remaining collection becomes half.

Comparison	Remaining Books
------------	-----------------

Start	16
-------	----

Comparison	Remaining Books
------------	-----------------

1st Comparison	8
----------------	---

2nd Comparison	4
----------------	---

3rd Comparison	2
----------------	---

4th Comparison	1
----------------	---

Thus,

$16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$

The search stops when only one book remains or the required title is found.

4. Calculation of Number of Steps

For Binary Search,

The number of comparisons is given by

$$\log_2 n$$

Suppose,

$$n = 16$$

Then,

$$\log_2 16 = 4$$

Therefore,

Only **4 comparisons** are needed to search among **16 books**.

Another example:

If

$$n = 1024$$

then

$$\log_2 1024 = 10$$

Only **10 comparisons** are needed to search among **1024 books**, making Binary Search very efficient.

5. Time Complexity

Since the search space is reduced by half after every comparison,

The recurrence relation is

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

After repeated halving,

$$n, \frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots, 1$$

The number of divisions is

$$\log_2 n$$

Therefore,

$\text{Time Complexity} = O(\log n)$

Time Complexity Table

Case	Time Complexity
Best Case	O(1) (Title found at the middle)
Average Case	O(log n)
Worst Case	O(log n)

6. Space Complexity

Recursive Binary Search

If Binary Search is implemented recursively, each recursive call is stored in the recursion stack.

The maximum recursion depth is

$$\log_2 n$$

Hence,

$$\text{Space Complexity} = O(\log n)$$

Iterative Binary Search

If Binary Search is implemented using a loop, no recursion stack is required.

Therefore,

$$\text{Space Complexity} = O(1)$$

7. Final Result

Algorithm Used

Binary Search (Divide-and-Conquer)

Problem Size Reduction

$$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \frac{n}{8} \rightarrow 1$$

Number of Steps

$$\log_2 n$$

Time Complexity

- Best Case: $O(1)$
- Average Case: $O(\log n)$
- Worst Case: $O(\log n)$

Space Complexity

- Recursive Binary Search: $O(\log n)$
- Iterative Binary Search: $O(1)$

Conclusion

An **E-Library Search System** uses the **Divide-and-Conquer strategy** to search for a book efficiently. By comparing the required title with the middle element, it

eliminates half of the remaining collection after each comparison. This repeated halving reduces the problem size from n to $n/2$, $n/4$, and so on, requiring only $\log_2 n$ comparisons. As a result, the search operation has a **time complexity of $O(\log n)$** . The **space complexity** is $O(\log n)$ for the recursive implementation due to the recursion stack, while it is $O(1)$ for the iterative implementation, making Binary Search an efficient technique for real-time e-library search systems.

1. Introduction (Real-Time Application)

An **Image Processing System** is used in applications such as **medical imaging, satellite image analysis, facial recognition, computer vision, and photo editing**. Large images are often divided into smaller parts so that they can be processed more efficiently.

For example, an image can be divided into **four equal quadrants**, and each quadrant is processed separately. After processing, the four parts are combined to produce the final image.

This divide-and-conquer approach is represented by the recurrence relation:

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

where:

- **T(n)** = Time required to process an image of size **n**
 - **4T(n/2)** = The image is divided into **4 equal sub-images**, each of size **n/2**
 - **+n** = Time required to combine or process the image at the current level
-

2. Meaning of the Recurrence Relation

The recurrence

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

indicates that:

- The original image is divided into **four equal parts**.
 - Each part is processed recursively.
 - After all four parts are processed, they are combined to obtain the final image.
 - This process continues until the image becomes small enough to process directly.
-

3. Applying the Master Theorem

The standard form of the Master Theorem is

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Comparing with the given recurrence,

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

we identify the parameters as:

Parameter Value Meaning

a	4	Number of recursive subproblems
b	2	Each subproblem size is reduced by half
f(n)	n	Time required to combine/process the image

4. Calculate $n^{\log_b a}$

Given,

$$a = 4, b = 2$$

Calculate,

$$\log_2 4 = 2$$

Therefore,

$$n^{\log_b a} = n^2$$

Now compare

$$f(n) = n$$

with

$$n^{\log_b a} = n^2$$

Since

$$n = O(n^{2-\varepsilon})$$

(where $\varepsilon = 1$),

we have

$$f(n) < n^{\log_b a}$$

5. Case Selection (Master Theorem)

According to the **Master Theorem**:

- **Case 1:** If

$$f(n) = O(n^{\log_b a - \varepsilon})$$

for some $\varepsilon > 0$,

then

$$T(n) = \Theta(n^{\log_b a})$$

Since

$$f(n) = n$$

and

$$n < n^2$$

Case 1 applies.

Therefore,

$$T(n) = \Theta(n^2)$$

Hence,

$$\text{Time Complexity} = O(n^2)$$

6. Performance Analysis with Image Size

As the image size increases, the algorithm divides the image into four equal parts at every recursive level.

Examples:

Image Size	Approximate Processing Time
Small Image (256 × 256)	Low
Medium Image (512 × 512)	Higher
Large Image (1024 × 1024)	Much Higher
Very Large Image (2048 × 2048)	Significantly Higher

Since the time complexity is

$$O(n^2)$$

the processing time increases **quadratically** with the image size.

For example:

- If the image size doubles, the processing time becomes approximately **four times larger**.
 - Therefore, larger images require much more processing time than smaller images.
-

7. Space Complexity

The recursion depth depends on how many times the image size can be divided by 2.

The recursion depth is

$$\log_2 n$$

Since only one recursive path is active at a time, the recursion stack stores approximately **log n** function calls.

Therefore,

$\text{Space Complexity} = O(\log n)$

8. Final Result

Recurrence Relation

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

Master Theorem Parameters

- **a = 4**
- **b = 2**
- **f(n) = n**

Calculation

$$n^{\log_2 4} = n^2$$

Since

$$f(n) = O(n^{2-1})$$

Master Theorem Case 1 applies.

Time Complexity

$$\boxed{O(n^2)}$$

Space Complexity

$$\boxed{O(\log n)}$$

Conclusion

An **Image Processing System** divides a large image into **four equal sub-images**, processes each recursively, and combines the results. This divide-and-conquer approach is represented by the recurrence relation

$$T(n) = 4T\left(\frac{n}{2}\right) + n.$$

Using the **Master Theorem**, the parameters are identified as **a = 4**, **b = 2**, and **f(n) = n**. Since $f(n)$ grows more slowly than $n^{\log_b a} = n^2$, **Case 1** of the Master Theorem applies. Therefore, the algorithm has a **time complexity of** $O(n^2)$. The recursion depth is $\log n$, giving a **space complexity of** $O(\log n)$. As the image size increases, the processing time increases **quadratically**, making larger images significantly more expensive to process than smaller ones.